

Helm Charts for Beginners (Kubernetes / OpenShift) — super simple

1) What Helm is (in plain language)

Think of **Helm** as an “app installer” for Kubernetes.

- Kubernetes runs your app using YAML files (Deployments, Services, etc.).
- A **Helm chart** is a **bundle** of those YAML files + a way to customize them.

So instead of managing 15 YAML files by hand, you say:

- “Install this chart”
 - “Use these settings (values)”
-

2) What a Helm chart contains (structure)

A typical chart looks like this:

```
`` mychart/ Chart.yaml values.yaml templates/ deployment.yaml
service.yaml ingress.yaml (or OpenShift Route) configmap.yaml charts/
(optional: dependencies) templates/_helpers.tpl (optional: reusable
template helpers) .helmignore (optional) ``
```

The key files

- `Chart.yaml`: chart metadata (name, version, description).
 - `values.yaml`: default configuration (image name, replicas, env vars, etc.).
 - `templates/`: Kubernetes YAML **templates**.
 - They are not plain YAML; they contain placeholders like `{{ .Values.image.repository }}`.
 - `charts/`: optional folder for dependent charts.
-

3) The Helm workflow (how people actually use it)

Step A — You have a chart You can get it from:

- a Git repo (source code)
- a Helm repository (like an “app store”)
- an OCI registry (container registry that can also store charts)

Step B — You choose your configuration (“values”) You can customize a chart in two common ways:

1) **Edit a values file** (recommended):

```
```yaml
```

## my-values.yaml

```
replicaCount: 2 image: repository: my-registry/my-app tag: "1.2.3" ```
```

2) **Override with --set** (good for quick changes):

```
```bash helm install myapp ./mychart --set replicaCount=2 ```
```

Step C — Helm renders templates into plain YAML This is the part your colleague called “compile”.

Helm doesn’t compile like Java/C++. It **renders** templates into final Kubernetes YAML.

You can see the rendered result without installing:

```
```bash helm template myapp ./mychart -f my-values.yaml ```
```

**Step D — Helm applies the YAML to the cluster** When you run install/upgrade, Helm sends the rendered YAML to the Kubernetes/OpenShift API:

```
```bash helm install myapp ./mychart -f my-values.yaml
```

later

```
helm upgrade myapp ./mychart -f my-values.yaml ```
```

Step E — Helm keeps “release state” Helm stores the release information in the cluster (usually as Secrets) in the namespace. That’s what enables:

- `helm list`
- `helm history`

- helm rollback
-

4) Common commands (cheat sheet)

Create a starter chart

```
``bash helm create mychart ``
```

Validate / lint the chart

```
``bash helm lint ./mychart ``
```

Render the chart ("compile") to YAML, no cluster needed

```
``bash helm template myapp ./mychart -f my-values.yaml ``
```

Install into the cluster

```
``bash helm install myapp ./mychart -f my-values.yaml ``
```

Upgrade

```
``bash helm upgrade myapp ./mychart -f my-values.yaml ``
```

Rollback

```
``bash helm rollback myapp 1 ``
```

5) “How do I get a Helm chart inside the OpenShift cluster?”

Very important clarification:

- You usually **don't** “upload the chart into the cluster” as a permanent object.
- What happens is:

1) Helm reads the chart **from your machine / CI runner** 2) Helm renders templates to YAML 3) Helm sends the YAML to the OpenShift API 4) The cluster stores the **release record** (so Helm can manage it later)

So the practical question is really:

“How do I run `helm install` against my OpenShift cluster?”

A) Connect Helm to OpenShift OpenShift is Kubernetes under the hood. Helm talks to it via your kubeconfig.

Typical flow:

1) Login: ```bash oc login https://api.<cluster>:6443 ```

2) Choose (or create) a project/namespace: ```bash oc new-project my-project`

or

`oc project my-project ```

3) Now Helm will use the same kubeconfig context: ```bash helm install myapp ./mychart -n my-project -f my-values.yaml ```

B) OpenShift specifics (what might break)

- **Ingress vs Route:**

- Kubernetes commonly uses Ingress.
- OpenShift often uses Route.
- Your chart may need to support Route templates (or support both).

- **Security / permissions:**

- OpenShift can be stricter (Security Context Constraints).
- A chart that works on vanilla Kubernetes may fail on OpenShift if it needs privileged settings.

- **ServiceAccount / RBAC:**

- Your chart may create Roles/RoleBindings.
- Make sure your user/CI has permission to create those objects.

C) “Where is the chart stored after install?”

- The chart package itself is not typically stored as a first-class object.
- Helm stores **release metadata** (and the rendered manifest) in the namespace as Secrets/ConfigMaps.

6) How teams deliver charts (real-world)

Option 1: Install from a folder in Git Common in internal teams:

```
``bash helm upgrade --install myapp ./charts/myapp -n my-project -f values-prod.yaml ``
```

Option 2: Install from a Helm repo Like an “app store”.

```
``bash helm repo add bitnami https://charts.bitnami.com/bitnami helm repo update helm install myredis bitnami/redis -n my-project ``
```

Option 3: Install from an OCI registry Store charts like container images.

```
``bash helm registry login my-registry helm pull oci://my-registry/charts/myapp --version 1.2.3 helm install myapp oci://my-registry/charts/myapp -n my-project ``
```

7) The simplest mental model

- **Chart** = reusable app blueprint
- **Values** = your configuration
- **Template rendering (“compile”)** = chart → final YAML
- **Install** = apply YAML to cluster + store release info

If you tell me what your colleague meant exactly by “get it inside the OpenShift cluster” (CI? chart repo? OpenShift UI?), I can add the exact recommended approach for your setup.